

Java集合面试题

 面试鸭 mianshiya.com

程序员面试 刷题神器

八股文一网打尽 面试从未如此轻松

- ✓ 9000+ 中大厂高频面试题
- ✓ 大厂面试官团队原创优质题解
- ✓ 网页版+小程序+IDE、VSCode插件
- ✓ 真题命中率高，持续更新

微信扫一扫 >>

刷高频题拿高薪offer



概念

数组与集合区别，用过哪些？

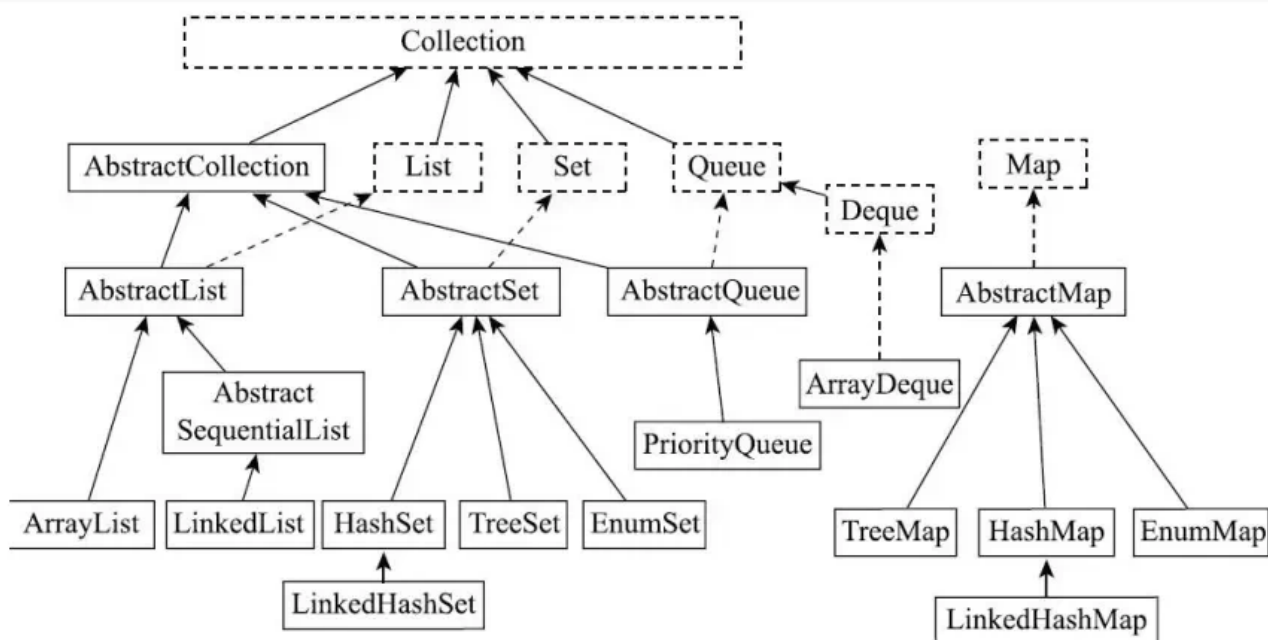
数组和集合的区别：

- 数组是固定长度的数据结构，一旦创建长度就无法改变，而集合是动态长度的数据结构，可以根据需要动态增加或减少元素。
- 数组可以包含基本数据类型和对象，而集合只能包含对象。
- 数组可以直接访问元素，而集合需要通过迭代器或其他方法访问元素。

我用过的一些 Java 集合类：

1. **ArrayList**：动态数组，实现了List接口，支持动态增长。
2. **LinkedList**：双向链表，也实现了List接口，支持快速的插入和删除操作。
3. **HashMap**：基于哈希表的Map实现，存储键值对，通过键快速查找值。
4. **HashSet**：基于HashMap实现的Set集合，用于存储唯一元素。
5. **TreeMap**：基于红黑树实现的有序Map集合，可以按照键的顺序进行排序。
6. **LinkedHashMap**：基于哈希表和双向链表实现的Map集合，保持插入顺序或访问顺序。
7. **PriorityQueue**：优先队列，可以按照比较器或元素的自然顺序进行排序。

说说Java中的集合？



List是有序的Collection，使用此接口能够精确的控制每个元素的插入位置，用户能根据索引访问List中元素。常用的实现List的类有LinkedList，ArrayList，Vector，Stack。

- ArrayList是容量可变的非线程安全列表，其底层使用数组实现。当几何扩容时，会创建更大的数组，并把原数组复制到新数组。ArrayList支持对元素的快速随机访问，但插入与删除速度很慢。
- LinkedList本质是一个双向链表，与ArrayList相比，其插入和删除速度更快，但随机访问速度更慢。

Set不允许存在重复的元素，与List不同，set中的元素是无序的。常用的实现有HashSet，LinkedHashSet和TreeSet。

- HashSet通过HashMap实现，HashMap的Key即HashSet存储的元素，所有Key都是用相同的Value，一个名为PRESENT的Object类型常量。使用Key保证元素唯一性，但不保证有序性。由于HashSet是HashMap实现的，因此线程不安全。
- LinkedHashSet继承自HashSet，通过LinkedHashMap实现，使用双向链表维护元素插入顺序。
- TreeSet通过TreeMap实现的，添加元素到集合时按照比较规则将其插入合适的位置，保证插入后的集合仍然有序。

Map 是一个键值对集合，存储键、值和之间的映射。Key 无序，唯一；value 不要求有序，允许重复。Map 没有继承于 Collection 接口，从 Map 集合中检索元素时，只要给出键对象，就会返回对应的值对象。主要实现有TreeMap、HashMap、HashTable、LinkedHashMap、ConcurrentHashMap

- HashMap: JDK1.8 之前 HashMap 由数组+链表组成的，数组是 HashMap 的主体，链表则是主要为了解决哈希冲突而存在的（“拉链法”解决冲突），JDK1.8 以后在解决哈希冲突时有了较大的变化，当链表长度大于阈值（默认为 8）时，将链表转化为红黑树，以减少搜索时间
- LinkedHashMap: LinkedHashMap 继承自 HashMap，所以它的底层仍然是基于拉链式散列结构即由数组和链表或红黑树组成。另外，LinkedHashMap 在上面结构的基础上，增加了一条双向链表，使得上面的结构可以保持键值对的插入顺序。同时通过对链表进行相应的操作，实现了访问顺序相关逻辑。
- HashTable: 数组+链表组成的，数组是 HashTable 的主体，链表则是主要为了解决哈希冲突而存在的
- TreeMap: 红黑树（自平衡的排序二叉树）
- ConcurrentHashMap: Node数组+链表+红黑树实现，线程安全的（jdk1.8以前Segment锁，1.8以后volatile + CAS 或者 synchronized）

Java中的线程安全的集合是什么？

在 java.util 包中的线程安全的类主要 2 个，其他都是非线程安全的。

- **Vector**：线程安全的动态数组，其内部方法基本都经过synchronized修饰，如果不需要线程安全，并不建议选择，毕竟同步是有额外开销的。Vector 内部是使用对象数组来保存数据，可以根据需要自动的增加容量，当数组已满时，会创建新的数组，并拷贝原有数组数据。
- **Hashtable**：线程安全的哈希表，Hashtable 的加锁方法是给每个方法加上 synchronized 关键字，这样锁住的是整个 Table 对象，不支持 null 键和值，由于同步导致的性能开销，所以已经很少被推荐使用，如果要保证线程安全的哈希表，可以用ConcurrentHashMap。

java.util.concurrent 包提供的都是线程安全的集合：

并发Map：

- **ConcurrentHashMap**：它与 Hashtable 的主要区别是二者加锁粒度的不同，在JDK1.7，ConcurrentHashMap加的是分段锁，也就是Segment锁，每个Segment 含有整个 table 的一部分，这样不同分段之间的并发操作就互不影响。在JDK 1.8，它取消了Segment字段，直接在table元素上加锁，实现对每一行进行加锁，进一步减小了并发冲突的概率。对于put操作，如果Key对应的数组元素为null，则通过CAS操作（Compare and Swap）将其设置为当前值。如果Key对应的数组元素（也即链表表头或者树的根元素）不为null，则对该元素使用 synchronized 关键字申请锁，然后进行操作。如果该 put 操作使得当前链表长度超过一定阈值，则将该链表转换为红黑树，从而提高寻址效率。
- **ConcurrentSkipListMap**：实现了一个基于SkipList（跳表）算法的可排序的并发集合，SkipList是一种可以在对数预期时间内完成搜索、插入、删除等操作的数据结构，通过维护多个指向其他元素的“跳跃”链接来实现高效查找。

并发Set：

- **ConcurrentSkipListSet**：是线程安全的有序的集合。底层是使用ConcurrentSkipListMap实现。
- **CopyOnWriteArraySet**：是线程安全的Set实现，它是线程安全的无序的集合，可以将它理解成线程安全的HashSet。有意思的是，CopyOnWriteArraySet和HashSet虽然都继承于共同的父类AbstractSet；但是，HashSet是通过“散列表”实现的，而CopyOnWriteArraySet则是通过“动态数组(CopyOnWriteArrayList)”实现的，并不是散列表。

并发List：

- **CopyOnWriteArrayList**：它是 ArrayList 的线程安全的变体，其中所有写操作（add，set等）都通过对底层数组进行全新复制来实现，允许存储 null 元素。即当对象进行写操作时，使用了Lock锁做同步处理，内部拷贝了原数组，并在新数组上进行添加操作，最后将新数组替换掉旧数组；若进行的读操作，则直接返回结果，操作过程中不需要进行同步。

并发 Queue：

- **ConcurrentLinkedQueue**：是一个适用于高并发场景下的队列，它通过无锁的方式(CAS)，实现了高并发状态下的高性能。通常，ConcurrentLinkedQueue 的性能要好于 BlockingQueue 。
- **BlockingQueue**：与 ConcurrentLinkedQueue 的使用场景不同，BlockingQueue 的主要功能并不是在于提升高并发时的队列性能，而在于简化多线程间的数据共享。BlockingQueue 提供一种读写阻塞等待的机制，即如果消费者速度较快，则 BlockingQueue 则可能被清空，此时消费线程再试图从 BlockingQueue 读取数据时就会被阻塞。反之，如果生产线程较快，则 BlockingQueue 可能会被装满，此时，生产线程再试图向 BlockingQueue 队列装入数据时，便会被阻塞等待。

并发 Deque：

- **LinkedBlockingDeque**：是一个线程安全的双端队列实现。它的内部使用链表结构，每一个节点都维护了一个前驱节点和一个后驱节点。LinkedBlockingDeque 没有进行读写锁的分离，因此同一时间只能有一个线程对其进行操作

- **ConcurrentLinkedDeque**: ConcurrentLinkedDeque是一种基于链接节点的无限并发链表。可以安全地并发执行插入、删除和访问操作。当许多线程同时访问一个公共集合时，ConcurrentLinkedDeque是一个合适的选择。

Collections和Collection的区别

- Collection是Java集合框架中的一个接口，它是所有集合类的基础接口。它定义了一组通用的操作和方法，如添加、删除、遍历等，用于操作和管理一组对象。Collection接口有许多实现类，如List、Set和Queue等。
- Collections（注意有一个s）是Java提供的一个工具类，位于java.util包中。它提供了一系列静态方法，用于对集合进行操作和算法。Collections类中的方法包括排序、查找、替换、反转、随机化等等。这些方法可以对实现了Collection接口的集合进行操作，如List和Set。

集合遍历的方法有哪些？

在Java中，集合的遍历方法主要有以下几种：

- **普通 for 循环**：可以使用带有索引的普通 for 循环来遍历 List。

```
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add("C");

for (int i = 0; i < list.size(); i++) {
    String element = list.get(i);
    System.out.println(element);
}
```

- **增强 for 循环 (for-each循环)**：用于循环访问数组或集合中的元素。

```
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add("C");

for (String element : list) {
    System.out.println(element);
}
```

- **Iterator 迭代器**：可以使用迭代器来遍历集合，特别适用于需要删除元素的情况。

```

List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add("C");

Iterator<String> iterator = list.iterator();
while(iterator.hasNext()) {
    String element = iterator.next();
    System.out.println(element);
}

```

- **ListIterator 列表迭代器：** ListIterator是迭代器的子类，可以双向访问列表并在迭代过程中修改元素。

```

List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add("C");

ListIterator<String> listIterator= list.listIterator();
while(listIterator.hasNext()) {
    String element = listIterator.next();
    System.out.println(element);
}

```

- **使用 forEach 方法：** Java 8引入了 forEach 方法，可以对集合进行快速遍历。

```

List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add("C");

list.forEach(element -> System.out.println(element));

```

- **Stream API：** Java 8的Stream API提供了丰富的功能，可以对集合进行函数式操作，如过滤、映射等。

```

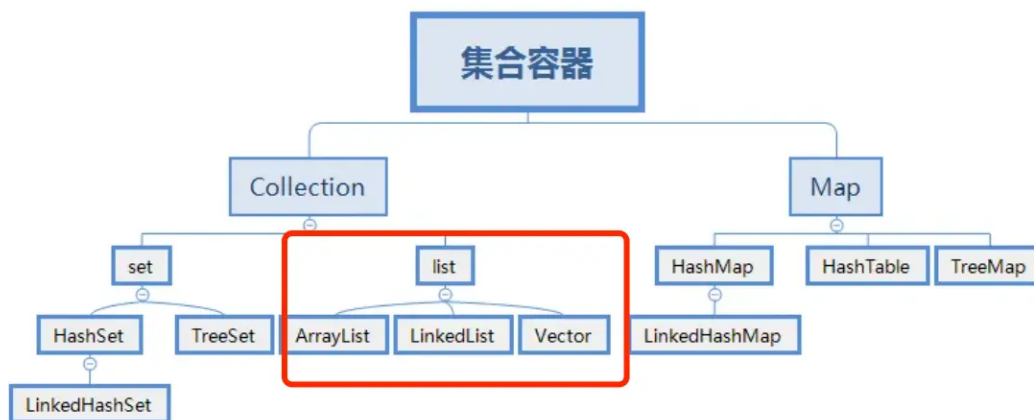
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add("C");

list.stream().forEach(element -> System.out.println(element));

```

这些是常用的集合遍历方法，根据情况选择合适的方法来遍历和操作集合。

List



常见的List集合（非线程安全）：

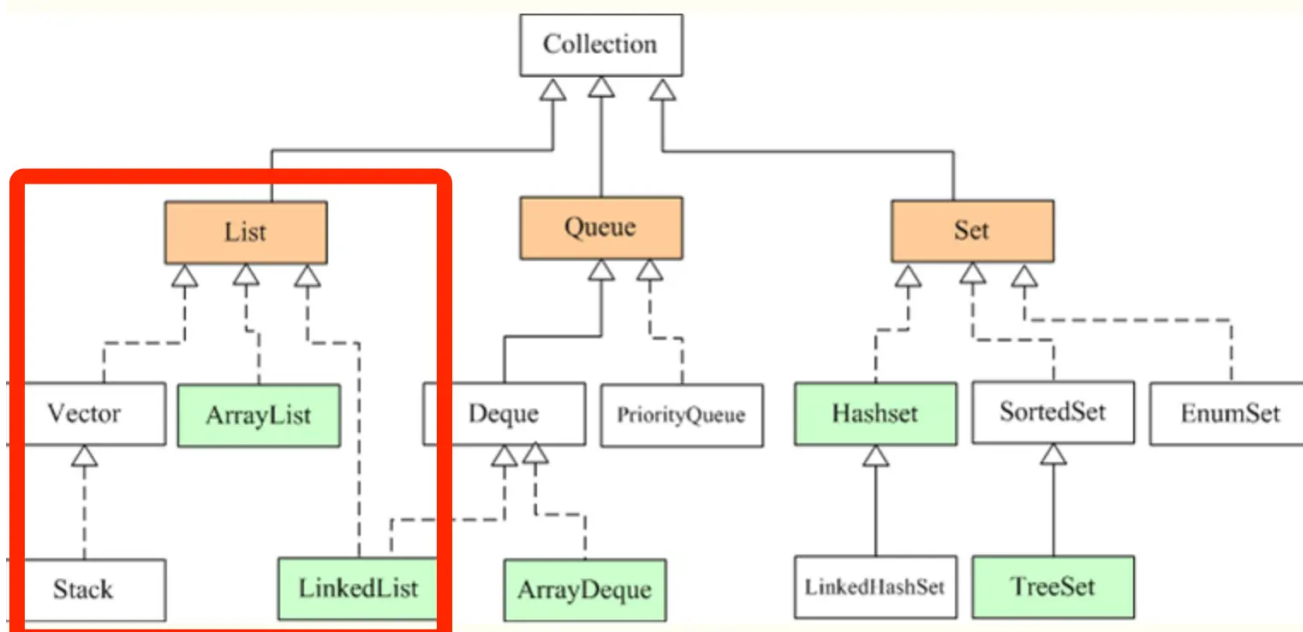
- `ArrayList` 基于动态数组实现，它允许快速的随机访问，即通过索引访问元素的时间复杂度为 $O(1)$ 。在添加和删除元素时，如果操作位置不是列表末尾，可能需要移动大量元素，性能相对较低。适用于需要频繁随机访问元素，而对插入和删除操作性能要求不高的场景，如数据的查询和展示等。
- `LinkedList` 基于双向链表实现，在插入和删除元素时，只需修改链表的指针，不需要移动大量元素，时间复杂度为 $O(1)$ 。但随机访问元素时，需要从链表头或链表尾开始遍历，时间复杂度为 $O(n)$ 。适用于需要频繁进行插入和删除操作的场景，如队列、栈等数据结构的实现，以及需要在列表中间频繁插入和删除元素的情况。

常见的List集合（线程安全）：

- `Vector` 和 `ArrayList` 类似，也是基于数组实现。`Vector` 中的方法大多是同步的，这使得它在多线程环境下可以保证数据的一致性，但在单线程环境下，由于同步带来的开销，性能会略低于 `ArrayList`。
- `CopyOnWriteArrayList` 在对列表进行修改（如添加、删除元素）时，会创建一个新的底层数组，将修改操作应用到新数组上，而读操作仍然在原数组上进行，这样可以保证读操作不会被写操作阻塞，实现了读写分离，提高了并发性能。适用于读操作远远多于写操作的并发场景，如事件监听列表等，在这种场景下可以避免大量的锁竞争，提高系统的性能和响应速度。

讲一下java里面list的几种实现，几种实现有什么不同？

在Java中，`List` 接口是最常用的集合类型之一，用于存储元素的有序集合。以下是Java中常见的 `List` 实现及其特点：



- `Vector` 是 Java 早期提供的线程安全的动态数组，如果不需要线程安全，并不建议选择，毕竟同步是有额外开销的。`Vector` 内部是使用对象数组来保存数据，可以根据需要自动的增加容量，当数组已满时，会创建新的数组，并拷贝原有数组数据。
- `ArrayList` 是应用更加广泛的动态数组实现，它本身不是线程安全的，所以性能要好很多。与 `Vector` 近似，`ArrayList` 也是可以根据需要调整容量，不过两者的调整逻辑有所区别，`Vector` 在扩容时会提高 1 倍，而 `ArrayList` 则是增加 50%。
- `LinkedList` 顾名思义是 Java 提供的双向链表，所以它不需要像上面两种那样调整容量，它也不是线程安全的。

这几种实现具体在什么场景下应该用哪种？

- `Vector` 和 `ArrayList` 作为动态数组，其内部元素以数组形式顺序存储的，所以非常适合随机访问的场合。除了尾部插入和删除元素，往往性能会相对较差，比如我们在中间位置插入一个元素，需要移动后续所有元素。
- 而 `LinkedList` 进行节点插入、删除却要高效得多，但是随机访问性能则要比动态数组慢。

list可以一边遍历一边修改元素吗？

在 Java 中，`List` 在遍历过程中是否可以修改元素取决于遍历方式和具体的 `List` 实现类，以下是几种常见情况：

- 使用普通for循环遍历：可以在遍历过程中修改元素，只要修改的索引不超出 `List` 的范围即可。

```
import java.util.ArrayList;
import java.util.List;

public class ListTraversalAndModification {
    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>();
        list.add(1);
        list.add(2);
        list.add(3);
```

// 使用普通for循环遍历并修改元素


```

        for (int i = 0; i < list.size(); i++) {
            list.set(i, list.get(i) * 2);
        }

        System.out.println(list);
    }
}

```

- 使用foreach循环遍历：一般不建议在 `foreach` 循环中直接修改正在遍历的 `List` 元素，因为这可能会导致意外的结果或 `ConcurrentModificationException` 异常。在 `foreach` 循环中修改元素可能会破坏迭代器的内部状态，因为 `foreach` 循环底层是基于迭代器实现的，在遍历过程中修改集合结构，会导致迭代器的预期结构和实际结构不一致。

```

import java.util.ArrayList;
import java.util.List;

public class ListTraversalAndModification {
    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>();
        list.add(1);
        list.add(2);
        list.add(3);

        // 使用foreach循环遍历并尝试修改元素，会抛出ConcurrentModificationException异常
        for (Integer num : list) {
            list.set(list.indexOf(num), num * 2);
        }

        System.out.println(list);
    }
}

```

- 使用迭代器遍历：可以使用迭代器的 `remove` 方法来删除元素，但如果要修改元素的值，需要通过迭代器的 `set` 方法来进行，而不是直接通过 `List` 的 `set` 方法，否则也可能会抛出 `ConcurrentModificationException` 异常。

```

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class ListTraversalAndModification {
    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>();
        list.add(1);
        list.add(2);
        list.add(3);

        // 使用迭代器遍历并修改元素
        Iterator<Integer> iterator = list.iterator();
        while (iterator.hasNext()) {
            Integer num = iterator.next();

```



```

        if (num == 2) {
            // 使用迭代器的set方法修改元素
            iterator.set(4);
        }
    }

    System.out.println(list);
}
}

```

对于线程安全的 `List`，如 `CopyOnWriteArrayList`，由于其采用了写时复制的机制，在遍历的同时可以进行修改操作，不会抛出 `ConcurrentModificationException` 异常，但可能会读取到旧的数据，因为修改操作是在新的副本上进行的。

list如何快速删除某个指定下标的元素？

`ArrayList` 提供了 `remove(int index)` 方法来删除指定下标的元素，该方法在删除元素后，会将后续元素向前移动，以填补被删除元素的位置。如果删除的是列表末尾的元素，时间复杂度为 $O(1)$ ；如果删除的是列表中间的元素，时间复杂度为 $O(n)$ ， n 为列表中元素的个数，因为需要移动后续的元素。示例代码如下：

```

import java.util.ArrayList;
import java.util.List;

public class ArrayListRemoveExample {
    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>();
        list.add(1);
        list.add(2);
        list.add(3);

        // 删除下标为1的元素
        list.remove(1);

        System.out.println(list);
    }
}

```

`LinkedList` 的 `remove(int index)` 方法也可以用来删除指定下标的元素。它需要先遍历到指定下标位置，然后修改链表的指针来删除元素。时间复杂度为 $O(n)$ ， n 为要删除元素的下标。不过，如果已知要删除的元素是链表的头节点或尾节点，可以直接通过修改头指针或尾指针来实现删除，时间复杂度为 $O(1)$ 。示例代码如下：

```

import java.util.LinkedList;
import java.util.List;

public class LinkedListRemoveExample {
    public static void main(String[] args) {
        List<Integer> list = new LinkedList<>();
        list.add(1);
        list.add(2);
        list.add(3);
    }
}

```

```

        // 删除下标为1的元素
        list.remove(1);

        System.out.println(list);
    }
}

```

`CopyOnWriteArrayList` 的 `remove` 方法同样可以删除指定下标的元素。由于 `CopyOnWriteArrayList` 在写操作时会创建一个新的数组，所以删除操作的时间复杂度取决于数组的复制速度，通常为 $O(n)$ ， n 为数组的长度。但在并发环境下，它的删除操作不会影响读操作，具有较好的并发性能。示例代码如下：

```

import java.util.concurrent.CopyOnWriteArrayList;

public class CopyOnWriteArrayListRemoveExample {
    public static void main(String[] args) {
        CopyOnWriteArrayList<Integer> list = new CopyOnWriteArrayList<>();
        list.add(1);
        list.add(2);
        list.add(3);

        // 删除下标为1的元素
        list.remove(1);

        System.out.println(list);
    }
}

```

ArrayList和LinkedList的区别，哪个集合是线程安全的？

ArrayList和LinkedList都是Java中常见的集合类，它们都实现了List接口。

- **底层数据结构不同**：ArrayList使用数组实现，通过索引进行快速访问元素。LinkedList使用链表实现，通过节点之间的指针进行元素的访问和操作。
- **插入和删除操作的效率不同**：ArrayList在尾部的插入和删除操作效率较高，但在中间或开头的插入和删除操作效率较低，需要移动元素。LinkedList在任意位置的插入和删除操作效率都比较高，因为只需要调整节点之间的指针，但是LinkedList是不支持随机访问的，所以除了头结点外插入和删除的时间复杂度都是 $O(n)$ ，效率也不是很高所以LinkedList基本没人用。
- **随机访问的效率不同**：ArrayList支持通过索引进行快速随机访问，时间复杂度为 $O(1)$ 。LinkedList需要从头或尾开始遍历链表，时间复杂度为 $O(n)$ 。
- **空间占用**：ArrayList在创建时需要分配一段连续的内存空间，因此会占用较大的空间。LinkedList每个节点只需要存储元素和指针，因此相对较小。
- **使用场景**：ArrayList适用于频繁随机访问和尾部的插入删除操作，而LinkedList适用于频繁的中间插入删除操作和不需要随机访问的场景。
- **线程安全**：这两个集合都不是线程安全的，Vector是线程安全的

ArrayList线程安全吗？把ArrayList变成线程安全有哪些方法？

不是线程安全的，ArrayList变成线程安全的方式有：

- 使用Collections类的synchronizedList方法将ArrayList包装成线程安全的List：

```
List<String> synchronizedList = Collections.synchronizedList(arrayList);
```

- 使用CopyOnWriteArrayList类代替ArrayList，它是一个线程安全的List实现：

```
CopyOnWriteArrayList<String> copyOnWriteArrayList = new CopyOnWriteArrayList<>(arrayList);
```

- 使用Vector类代替ArrayList，Vector是线程安全的List实现：

```
Vector<String> vector = new Vector<>(arrayList);
```

为什么ArrayList不是线程安全的，具体来说哪里不安全？

在高并发添加数据下，ArrayList会暴露三个问题：

- 部分值为null（我们并没有add null进去）
- 索引越界异常
- size与我们add的数量不符

为了知道这三种情况是怎么发生的，ArrayList，add 增加元素的代码如下：

```
public boolean add(E e) {  
    ensureCapacityInternal(size + 1); // Increments modCount!!  
    elementData[size++] = e;  
    return true;  
}
```

ensureCapacityInternal()这个方法的详细代码我们可以暂时不看，它的作用就是判断如果将当前的新元素加到列表后面，列表的elementData数组的大小是否满足，如果size + 1的这个需求长度大于了elementData这个数组的长度，那么就要对这个数组进行扩容。

大体可以分为三步：

- 判断数组需不需要扩容，如果需要的话，调用grow方法进行扩容；
- 将数组的size位置设置值（因为数组的下标是从0开始的）；
- 将当前集合的大小加1

下面我们来分析三种情况都是如何产生的：

- 部分值为null：当线程1走到了扩容那里发现当前size是9，而数组容量是10，所以不用扩容，这时候cpu让出执行权，线程2也进来了，发现size是9，而数组容量是10，所以不用扩容，这时候线程1继续执行，将数组下标索引为9的位置set值了，还没有来得及执行size++，这时候线程2也来执行了，又把数组下标索引为9的位置set了一遍，这时候两个先后进行size++，导致下标索引10的地方就为null了。
- 索引越界异常：线程1走到扩容那里发现当前size是9，数组容量是10不用扩容，cpu让出执行权，线程2也发现不用扩容，这时候数组的容量就是10，而线程1 set完之后size++，这时候线程2再进来size就是10，数组的大小只有10，而你要设置下标索引为10的就会越界（数组的下标索引从0开始）；
- size与我们add的数量不符：这个基本上每次都会发生，这个理解起来也很简单，因为size++本身就不是原子操作，可以分为三步：获取size的值，将size的值加1，将新的size值覆盖掉原来的，线程1和线程2拿到一样的size值加完了同时覆盖，就会导致一次没有加上，所以肯定不会与我们add的数量保持一致的；

ArrayList 和 LinkedList 的应用场景？

- ArrayList适用于需要频繁访问集合元素的场景。它基于数组实现，可以通过索引快速访问元素，因此在按索引查找、遍历和随机访问元素的操作上具有较高的性能。当需要频繁访问和遍历集合元素，并且集合大小不经常改变时，推荐使用ArrayList
- LinkedList适用于频繁进行插入和删除操作的场景。它基于链表实现，插入和删除元素的操作只需要调整节点的指针，因此在插入和删除操作上具有较高的性能。当需要频繁进行插入和删除操作，或者集合大小经常改变时，可以考虑使用LinkedList。

ArrayList的扩容机制说一下

ArrayList在添加元素时，如果当前元素个数已经达到了内部数组的容量上限，就会触发扩容操作。ArrayList的扩容操作主要包括以下几个步骤：

- 计算新的容量：一般情况下，新的容量会扩大为原容量的1.5倍（在JDK 10之后，扩容策略做了调整），然后检查是否超过了最大容量限制。
- 创建新的数组：根据计算得到的新容量，创建一个新的更大的数组。
- 将元素复制：将原来数组中的元素逐个复制到新数组中。
- 更新引用：将ArrayList内部指向原数组的引用指向新数组。
- 完成扩容：扩容完成后，可以继续添加新元素。

ArrayList的扩容操作涉及到数组的复制和内存的重新分配，所以在频繁添加大量元素时，扩容操作可能会影响性能。为了减少扩容带来的性能损耗，可以在初始化ArrayList时预分配足够大的容量，避免频繁触发扩容操作。

之所以扩容是 1.5 倍，是因为 1.5 可以充分利用移位操作，减少浮点数或者运算时间和运算次数。

```
// 新容量计算
int newCapacity = oldCapacity + (oldCapacity >> 1);
```

线程安全的 List，CopyOnWriteArraylist是如何实现线程安全的

CopyOnWriteArrayList底层也是通过一个数组保存数据，使用volatile关键字修饰数组，保证当前线程对数组对象重新赋值后，其他线程可以及时感知到。

```
private transient volatile Object[] array;
```

在写入操作时，加了一把互斥锁ReentrantLock以保证线程安全。

```
public boolean add(E e) {
    //获取锁
    final ReentrantLock lock = this.lock;
    //加锁
    lock.lock();
    try {
        //获取到当前List集合保存数据的数组
        Object[] elements = getArray();
        //获取该数组的长度（这是一个伏笔，同时len也是新数组的最后一个元素的索引值）
        int len = elements.length;
        //将当前数组拷贝一份的同时，让其长度加1
        Object[] newElements = Arrays.copyOf(elements, len + 1);
```

//将加入的元素放在新数组最后一位，len不是旧数组长度吗，为什么现在用它当成新数组的最后一个元素的下标？建议自行画图推演，就很容易理解。

```
newElements[len] = e;
//替换引用，将数组的引用指向给新数组的地址
setArray(newElements);
return true;
} finally {
    //释放锁
    lock.unlock();
}
```

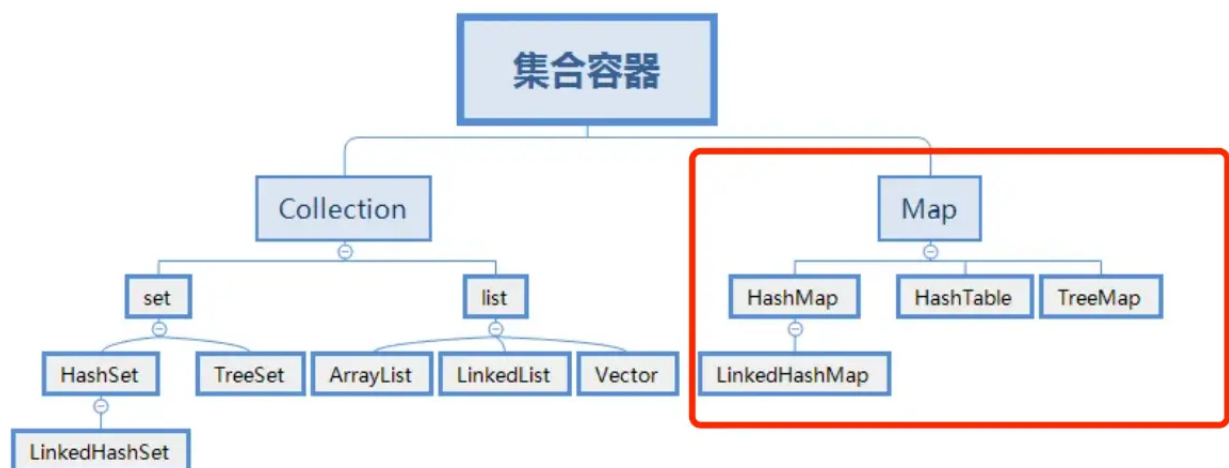
看到源码可以知道写入新元素时，首先会先将原来的数组拷贝一份并且让原来数组的长度+1后就得到了一个新数组，新数组里的元素和旧数组的元素一样并且长度比旧数组多一个长度，然后将新加入的元素放置都在新数组最后一个位置后，用新数组的地址替换掉老数组的地址就能得到最新的数据了。

在我们执行替换地址操作之前，读取的是老数组的数据，数据是有效数据；执行替换地址操作之后，读取的是新数组的数据，同样也是有效数据，而且使用该方式能比读写都加锁要更加的效率。

现在我们来查看读操作，读是没有加锁的，所以读是一直都能读

```
public E get(int index) {
    return get(getArray(), index);
}
```

Map



常见的Map集合（非线程安全）：

- `HashMap` 是基于哈希表实现的 `Map`，它根据键的哈希值来存储和获取键值对，JDK 1.8中是用数组+链表+红黑树来实现的。`HashMap` 是非线程安全的，在多线程环境下，当多个线程同时对 `HashMap` 进行操作时，可能会导致数据不一致或出现死循环等问题。比如在扩容时，多个线程可能会同时修改哈希表的结构，从而破坏数据的完整性。

- `LinkedHashMap` 继承自 `HashMap`，它在 `HashMap` 的基础上，使用双向链表维护了键值对的插入顺序或访问顺序，使得迭代顺序与插入顺序或访问顺序一致。由于它继承自 `HashMap`，在多线程并发访问时，同样会出现与 `HashMap` 类似的线程安全问题。
- `TreeMap` 是基于红黑树实现的 `Map`，它可以对键进行排序，默认按照自然顺序排序，也可以通过指定的比较器进行排序。`TreeMap` 是非线程安全的，在多线程环境下，如果多个线程同时对 `TreeMap` 进行插入、删除等操作，可能会破坏红黑树的结构，导致数据不一致或程序出现异常。

常见的Map集合（线程安全）：

- `Hashtable` 是早期 Java 提供的线程安全的 `Map` 实现，它的实现方式与 `HashMap` 类似，但在方法上使用了 `synchronized` 关键字来保证线程安全。通过在每个可能修改 `Hashtable` 状态的方法上加上 `synchronized` 关键字，使得在同一时刻，只能有一个线程能够访问 `Hashtable` 的这些方法，从而保证了线程安全。
- `ConcurrentHashMap` 在 JDK 1.8 以前采用了分段锁等技术来提高并发性能。在 `ConcurrentHashMap` 中，将数据分成多个段（Segment），每个段都有自己的锁。在进行插入、删除等操作时，只需要获取相应段的锁，而不是整个 `Map` 的锁，这样可以允许多个线程同时访问不同的段，提高了并发访问的效率。在 JDK 1.8 以后是通过 `volatile` + `CAS` 或者 `synchronized` 来保证线程安全的。

如何对map进行快速遍历？

- 使用for-each循环和`entrySet()`方法：这是一种较为常见和简洁的遍历方式，它可以同时获取 `Map` 中的键和值

```
import java.util.HashMap;
import java.util.Map;

public class MapTraversalExample {
    public static void main(String[] args) {
        Map<String, Integer> map = new HashMap<>();
        map.put("key1", 1);
        map.put("key2", 2);
        map.put("key3", 3);

        // 使用for-each循环和entrySet()遍历Map
        for (Map.Entry<String, Integer> entry : map.entrySet()) {
            System.out.println("Key: " + entry.getKey() + ", Value: " + entry.getValue());
        }
    }
}
```

- 使用for-each循环和`keySet()`方法：如果只需要遍历 `Map` 中的键，可以使用 `keySet()` 方法，这种方式相对简单，性能也较好。

```
import java.util.HashMap;
import java.util.Map;

public class MapTraversalExample {
    public static void main(String[] args) {
        Map<String, Integer> map = new HashMap<>();
        map.put("key1", 1);
        map.put("key2", 2);
        map.put("key3", 3);
    }
}
```

```
// 使用for-each循环和keySet()遍历Map的键
for (String key : map.keySet()) {
    System.out.println("Key: " + key + ", Value: " + map.get(key));
}
}
```

- 使用迭代器：通过获取Map的entrySet()或keySet()的迭代器，也可以实现对Map的遍历，这种方式在需要删除元素等操作时比较有用。

```
import java.util.HashMap;
import java.util.Iterator;
import java.util.Map;
import java.util.Map.Entry;

public class MapTraversalExample {
    public static void main(String[] args) {
        Map<String, Integer> map = new HashMap<>();
        map.put("key1", 1);
        map.put("key2", 2);
        map.put("key3", 3);

        // 使用迭代器遍历Map
        Iterator<Entry<String, Integer>> iterator = map.entrySet().iterator();
        while (iterator.hasNext()) {
            Entry<String, Integer> entry = iterator.next();
            System.out.println("Key: " + entry.getKey() + ", Value: " + entry.getValue());
        }
    }
}
```

- 使用 Lambda 表达式和forEach()方法：在 Java 8 及以上版本中，可以使用 Lambda 表达式和 `forEach()` 方法来遍历 `Map`，这种方式更加简洁和函数式。

```
import java.util.HashMap;
import java.util.Map;

public class MapTraversalExample {
    public static void main(String[] args) {
        Map<String, Integer> map = new HashMap<>();
        map.put("key1", 1);
        map.put("key2", 2);
        map.put("key3", 3);

        // 使用Lambda表达式和forEach()方法遍历Map
        map.forEach((key, value) -> System.out.println("Key: " + key + ", Value: " + value));
    }
}
```


- 使用Stream API: Java 8 引入的 `Stream API` 也可以用于遍历 `Map`，可以将 `Map` 转换为流，然后进行各种操作。

```
import java.util.HashMap;
import java.util.Map;
import java.util.stream.Collectors;

public class MapTraversalExample {
    public static void main(String[] args) {
        Map<String, Integer> map = new HashMap<>();
        map.put("key1", 1);
        map.put("key2", 2);
        map.put("key3", 3);

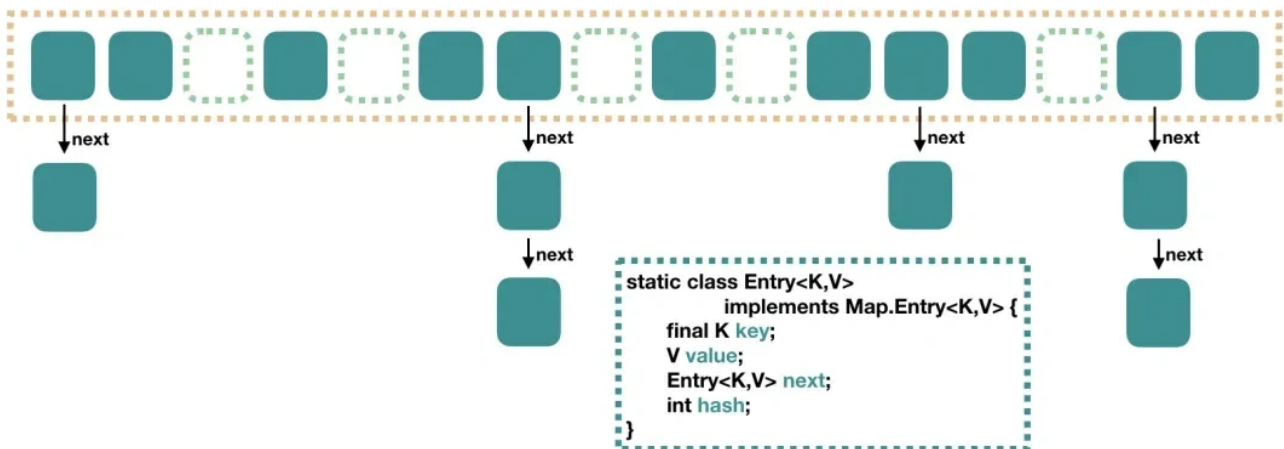
        // 使用Stream API遍历Map
        map.entrySet().stream()
            .forEach(entry -> System.out.println("Key: " + entry.getKey() + ", Value: " +
            entry.getValue()));

        // 还可以进行其他操作, 如过滤、映射等
        Map<String, Integer> filteredMap = map.entrySet().stream()
            .filter(entry -> entry.getValue() > 1)
            .collect(Collectors.toMap(Map.Entry::getKey,
            Map.Entry::getValue));
        System.out.println(filteredMap);
    }
}
```

HashMap实现原理介绍一下?

在JDK 1.7 版本之前，HashMap 数据结构是数组和链表，HashMap通过哈希算法将元素的键（Key）映射到数组中的槽位（Bucket）。如果多个键映射到同一个槽位，它们会以链表的形式存储在同一个槽位上，因为链表的查询时间是 $O(n)$ ，所以冲突很严重，一个索引上的链表非常长，效率就很低了。

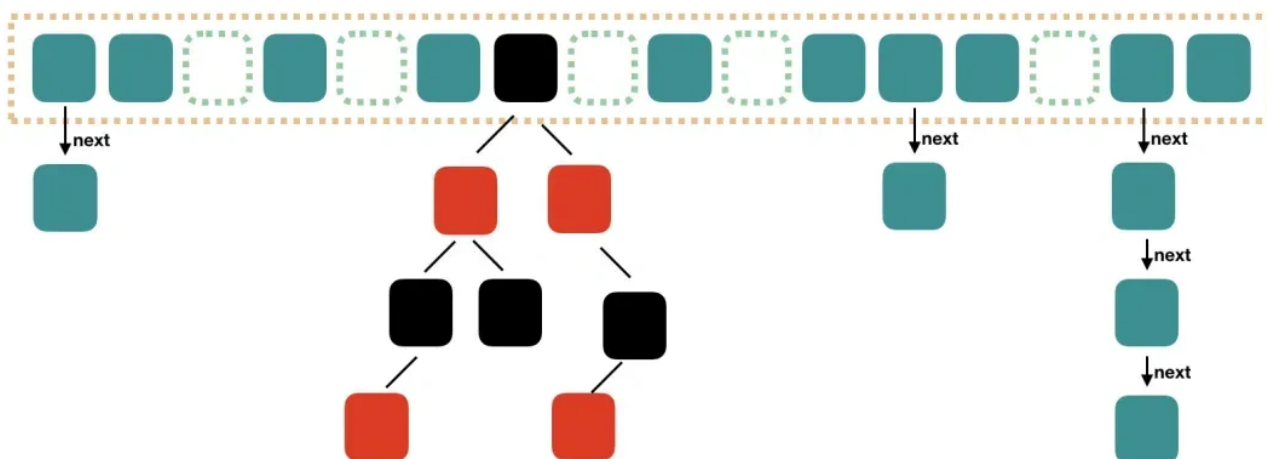
Java7 HashMap 结构



所以在JDK 1.8 版本的时候做了优化，当一个链表的长度超过8的时候就转换数据结构，不再使用链表存储，而是

使用**红黑树**，查找时使用红黑树，时间复杂度 $O(\log n)$ ，可以提高查询性能，但是在数量较少时，即数量小于6时，会将红黑树转换回链表。

Java8 HashMap 结构



了解的哈希冲突解决方法有哪些？

- 链接法：使用链表或其他数据结构来存储冲突的键值对，将它们链接在同一个哈希桶中。
- 开放寻址法：在哈希表中找到另一个可用的位置来存储冲突的键值对，而不是存储在链表中。常见的开放寻址方法包括线性探测、二次探测和双重散列。
- 再哈希法（Rehashing）：当发生冲突时，使用另一个哈希函数再次计算键的哈希值，直到找到一个空槽来存储键值对。
- 哈希桶扩容：当哈希冲突过多时，可以动态地扩大哈希桶的数量，重新分配键值对，以减少冲突的概率。

HashMap是线程安全的吗？

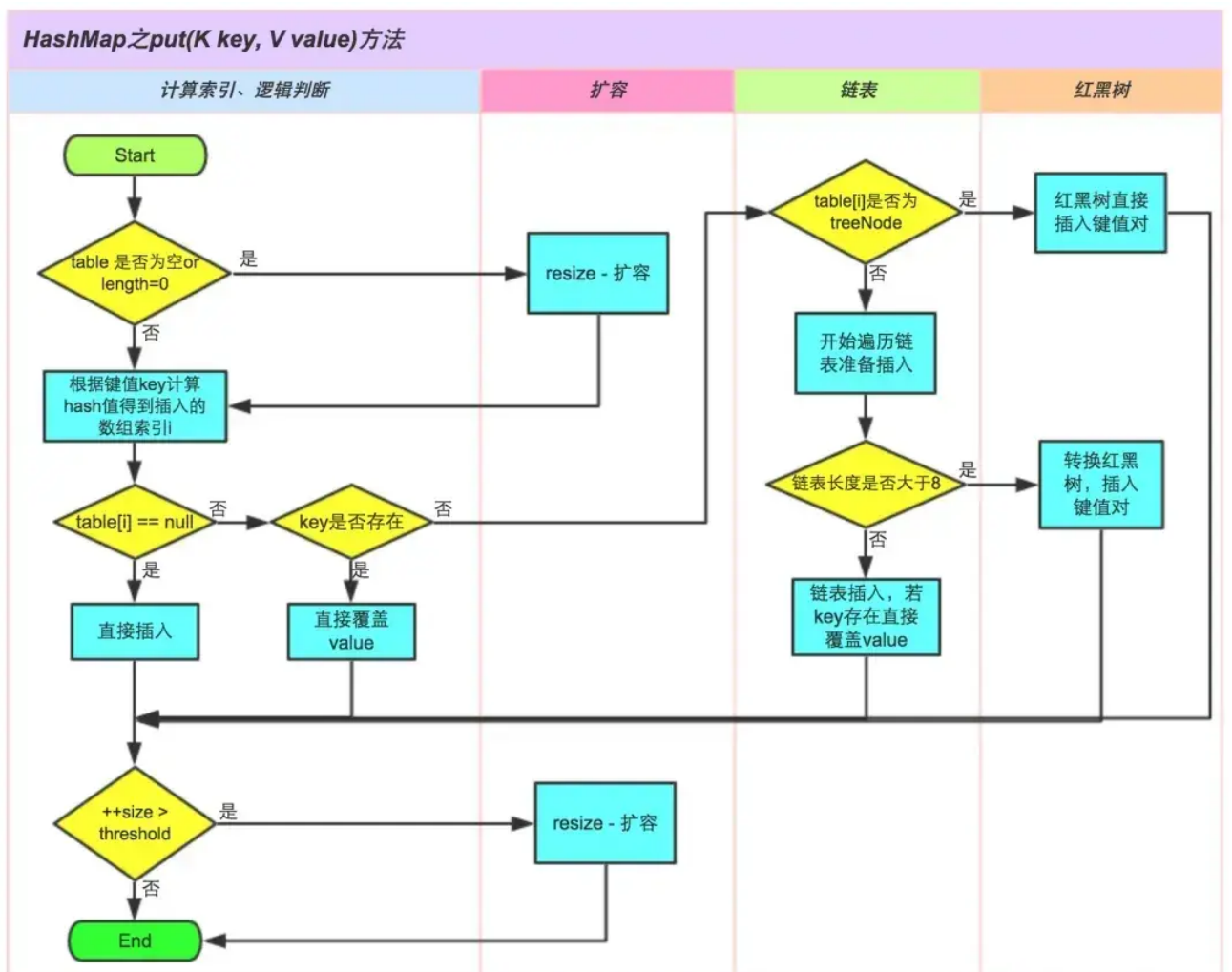
hashmap不是线程安全的，hashmap在多线程会存在下面的问题：

- JDK 1.7 HashMap 采用数组 + 链表的数据结构，多线程背景下，在数组扩容的时候，存在 Entry 链死循环和数据丢失问题。
- JDK 1.8 HashMap 采用数组 + 链表 + 红黑二叉树的数据结构，优化了 1.7 中数组扩容的方案，解决了 Entry 链死循环和数据丢失问题。但是多线程背景下，put 方法存在数据覆盖的问题。

如果要保证线程安全，可以通过这些方法来保证：

- 多线程环境可以使用Collections.synchronizedMap同步加锁的方式，还可以使用HashTable，但是同步的方式显然性能不达标，而ConcurrentHashMap更适合高并发场景使用。
- ConcurrentHashMap在JDK1.7和1.8的版本改动比较大，1.7使用Segment+HashEntry分段锁的方式实现，1.8则抛弃了Segment，改为使用CAS+synchronized+Node实现，同样也加入了红黑树，避免链表过长导致性能的问题。

hashmap的put过程介绍一下



HashMap HashMap的put()方法用于向HashMap中添加键值对，当调用HashMap的put()方法时，会按照以下详细流程执行（JDK8 1.8版本）：

第一步：根据要添加的键的哈希码计算在数组中的位置（索引）。

第二步：检查该位置是否为空（即没有键值对存在）

- 如果为空，则直接在该位置创建一个新的Entry对象来存储键值对。将要添加的键值对作为该Entry的键和值，并保存在数组的对应位置。将HashMap的修改次数（modCount）加1，以便在进行迭代时发现并修改。

第三步：如果该位置已经存在其他键值对，检查该位置的第一个键值对的哈希码和键是否与要添加的键值对相同？

- 如果相同，则表示找到了相同的键，直接将新的值替换旧的值，完成更新操作。

第四步：如果第一个键值对的哈希码和键不相同，则需要遍历链表或红黑树来查找是否有相同的键：

如果键值对集合是链表结构，从链表的头部开始逐个比较键的哈希码和equals()方法，直到找到相同的键或达到链表末尾。

- 如果找到了相同的键，则使用新的值取代旧的值，即更新键对应的值。
- 如果没有找到相同的键，则将新的键值对添加到链表的头部。

如果键值对集合是红黑树结构，在红黑树中使用哈希码和equals()方法进行查找。根据键的哈希码，定位到红黑树中的某个节点，然后逐个比较键，直到找到相同的键或达到红黑树末尾。

- 如果找到了相同的键，则使用新的值取代旧的值，即更新键对应的值。
- 如果没有找到相同的键，则将新的键值对添加到红黑树中。

第五步：检查链表长度是否达到阈值（默认为8）：

- 如果链表长度超过阈值，且HashMap的数组长度大于等于64，则会将链表转换为红黑树，以提高查询效率。

第六步：检查负载因子是否超过阈值（默认为0.75）：

- 如果键值对的数量（size）与数组的长度的比值大于阈值，则需要扩容操作。

第七步：扩容操作：

- 创建一个新的两倍大小的数组。
- 将旧数组中的键值对重新计算哈希码并分配到新数组中的位置。
- 更新HashMap的数组引用和阈值参数。

第八步：完成添加操作。

此外，HashMap是非线程安全的，如果在多线程环境下使用，需要采取额外的同步措施或使用线程安全的ConcurrentHashMap。

HashMap的put(key,val)和get(key)过程

- 存储对象时，我们将K/V传给put方法时，它调用hashCode计算hash从而得到bucket位置，进一步存储，HashMap会根据当前bucket的占用情况自动调整容量(超过Load Factor则resize为原来的2倍)。
- 获取对象时，我们将K传给get，它调用hashCode计算hash从而得到bucket位置，并进一步调用equals()方法确定键值对。如果发生碰撞的时候，HashMap通过链表将产生碰撞冲突的元素组织起来，在Java 8中，如果一个bucket中碰撞冲突的元素超过某个限制(默认是8)，则使用红黑树来替换链表，从而提高速度。

hashmap 调用get方法一定安全吗？

不是，调用 get 方法有几点需要注意的地方：

- **空指针异常 (NullPointerException)**：如果你尝试用 `null` 作为键调用 `get` 方法，而 `HashMap` 没有被初始化（即为 `null`），那么会抛出空指针异常。不过，如果 `HashMap` 已经初始化，使用 `null` 作为键是允许的，因为 `HashMap` 支持 `null` 键。
- **线程安全**：`HashMap` 本身不是线程安全的。如果在多线程环境中，没有适当的同步措施，同时对 `HashMap` 进行读写操作可能会导致不可预测的行为。例如，在一个线程中调用 `get` 方法读取数据，而另一个线程同时修改了结构（如增加或删除元素），可能会导致读取操作得到错误的结果或抛出 `ConcurrentModificationException`。如果需要在多线程环境中使用类似 `HashMap` 的数据结构，可以考虑使用 `ConcurrentHashMap`。

HashMap一般用什么做Key？为啥String适合做Key呢？

用 string 做 key，因为 String 对象是不可变的，一旦创建就不能被修改，这确保了Key的稳定性。如果Key是可变的，可能会导致hashCode和equals方法的不一致，进而影响HashMap的正确性。

为什么HashMap要用红黑树而不是平衡二叉树？

- 平衡二叉树追求的是一种“完全平衡”状态：任何结点的左右子树的高度差不会超过 1，优势是树的结点是很平均分配的。这个要求实在是太严了，导致每次进行插入/删除节点的时候，几乎都会破坏平衡树的第二个规则，进而我们都需要通过左旋和右旋来进行调整，使之再次成为一颗符合要求的平衡树。

- 红黑树不追求这种完全平衡状态，而是追求一种“弱平衡”状态：整个树最长路径不会超过最短路径的 2 倍。优势是虽然牺牲了一部分查找的性能效率，但是能够换取一部分维持树平衡状态的成本。与平衡树不同的是，红黑树在插入、删除等操作，**不会像平衡树那样，频繁着破坏红黑树的规则，所以不需要频繁着调整**，这也是我们为什么大多数情况下使用红黑树的原因。

hashmap key可以为null吗？

可以为 null。

- hashMap中使用hash()方法来计算key的哈希值，当key为空时，直接另key的哈希值为0，不走key.hashCode()方法；

```
static final int hash(Object key) {  
    int h;  
    // 当key等于null的时候，不走hashCode() 方法  
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);  
}
```

- hashMap虽然支持key和value为null，但是null作为key只能有一个，null作为value可以有多个；
- 因为hashMap中，如果key值一样，那么会覆盖相同key值的value为最新，所以key为null只能有一个。

重写HashMap的equal和hashCode方法需要注意什么？

HashMap使用Key对象的hashCode()和equals方法去决定key-value对的索引。当我们试着从HashMap中获取值的时候，这些方法也会被用到。如果这些方法没有被正确地实现，在这种情况下，两个不同Key也许会产生相同的hashCode()和equals()输出，HashMap将会认为它们是相同的，然后覆盖它们，而非把它们存储到不同的地方。

同样的，所有不允许存储重复数据的集合类都使用hashCode()和equals()去查找重复，所以正确实现它们非常重要。equals()和hashCode()的实现应该遵循以下规则：

- 如果o1.equals(o2)，那么o1.hashCode() == o2.hashCode()总是为true的。
- 如果o1.hashCode() == o2.hashCode()，并不意味着o1.equals(o2)会为true。

重写HashMap的equal方法不当会出现什么问题？

HashMap在比较元素时，会先通过hashCode进行比较，相同的情况下再通过equals进行比较。

所以 equals相等的两个对象，hashCode一定相等。hashCode相等的两个对象，equals不一定相等（比如散列冲突的情况）

重写了equals方法，不重写hashCode方法时，可能会出现equals方法返回为true，而hashCode方法却返回false，这样的后果会导致在hashmap等类中存储多个一模一样的对象，导致出现覆盖存储的数据的问题，这与hashmap只能有唯一的key的规范不符合。

列举HashMap在多线程下可能会出现的问题？

- JDK1.7中的 HashMap 使用头插法插入元素，在多线程的环境下，扩容的时候有可能导致环形链表的出现，形成死循环。因此，JDK1.8使用尾插法插入元素，在扩容时会保持链表元素原本的顺序，不会出现环形链表的问题。
- 多线程同时执行 put 操作，如果计算出来的索引位置是相同的，那会造成前一个 key 被后一个 key 覆盖，从而导致元素的丢失。此问题在JDK 1.7和JDK 1.8 中都存在。

HashMap的扩容机制介绍一下

hashMap默认的负载因子是0.75，即如果hashmap中的元素个数超过了总容量75%，则会触发扩容，扩容分为两个步骤：

- **第1步**是对哈希表长度的扩展（2倍）
- **第2步**是将旧哈希表中的数据放到新的哈希表中。

因为我们使用的是2次幂的扩展(指长度扩为原来2倍)，所以，元素的位置要么是在原位置，要么是在原位置再移动2次幂的位置。

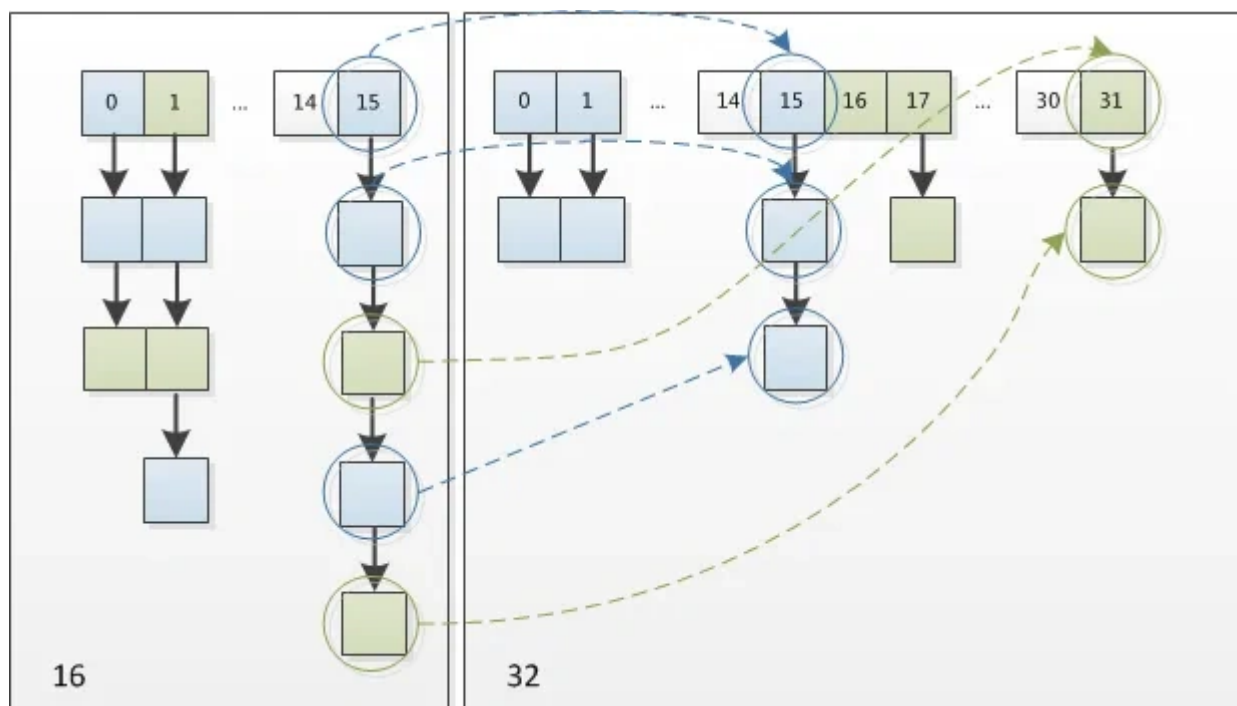
如我们从16扩展为32时，具体的变化如下所示：

```
n - 1  0000 0000 0000 0000 0000 0000 0000 1111
hash1  1111 1111 1111 1111 0000 1111 0000 0101  →  1111 1111 1111 1111 0000 1111 0000 1111
hash2  1111 1111 1111 1111 0000 1111 0001 0101      1111 1111 1111 1111 0000 1111 0001 0101
```

因此元素在重新计算hash之后，因为n变为2倍，那么n-1的mask范围在高位多1bit(红色)，因此新的index就会发生这样的变化：

$0101 = 5 \xrightarrow[16 \Rightarrow 2 \times 16]{\text{resize}} \begin{matrix} 00101 = 5 & \text{原位置} \\ 10101 = 21 = 5 + 16 & \text{原位置} + \text{oldCap} \end{matrix}$

因此，我们在扩充HashMap的时候，不需要重新计算hash，只需要看看原来的hash值新增的那个bit是1还是0就好了，是0的话索引没变，是1的话索引变成“原索引+oldCap”。可以看看下图为16扩充为32的resize示意图：



这个设计确实非常的巧妙，既省去了重新计算hash值的时间，而且同时，由于新增的1bit是0还是1可以认为是随机的，因此resize的过程，均匀的把之前的冲突的节点分散到新的bucket了。

HashMap的大小为什么是2的n次方大小呢？

在JDK1.7中，HashMap整个扩容过程就是分别取出数组元素，一般该元素是最后一个放入链表中的元素，然后遍历以该元素为头的单向链表元素，依据每个被遍历元素的hash值计算其在新数组中的下标，然后进行交换。这样的扩容方式会将原来哈希冲突的单向链表尾部变成扩容后单向链表的头部。

而在JDK 1.8中，HashMap对扩容操作做了优化。由于扩容数组的长度是2倍关系，所以对于假设初始tableSize = 4要扩容到8来说就是0100到1000的变化（左移一位就是2倍），在扩容中只用判断原来的hash值和左移动的一位（newtable的值）按位与操作是0或1就行，0的话索引不变，1的话索引变成原索引加上扩容前数组。

之所以能通过这种“与运算”来重新分配索引，是因为hash值本来就是随机的，而hash按位与上newTable得到的0（扩容前的索引位置）和1（扩容前索引位置加上扩容前数组长度的数值索引处）就是随机的，所以扩容的过程就能把之前哈希冲突的元素再随机分布到不同的索引中去。

往hashmap存20个元素，会扩容几次？

当插入20个元素时，HashMap的扩容过程如下：

初始容量：16

- 插入第1到第12个元素时，不需要扩容。
- 插入第13个元素时，达到负载因子限制，需要扩容。此时，HashMap的容量从16扩容到32。

扩容后的容量：32

- 插入第14到第24个元素时，不需要扩容。

因此，总共会进行一次扩容。

说说hashmap的负载因子

HashMap负载因子loadFactor的默认值是0.75，当HashMap中的元素个数超过了容量的75%时，就会进行扩容。

默认负载因子为0.75，是因为它提供了空间和时间复杂度之间的良好平衡。

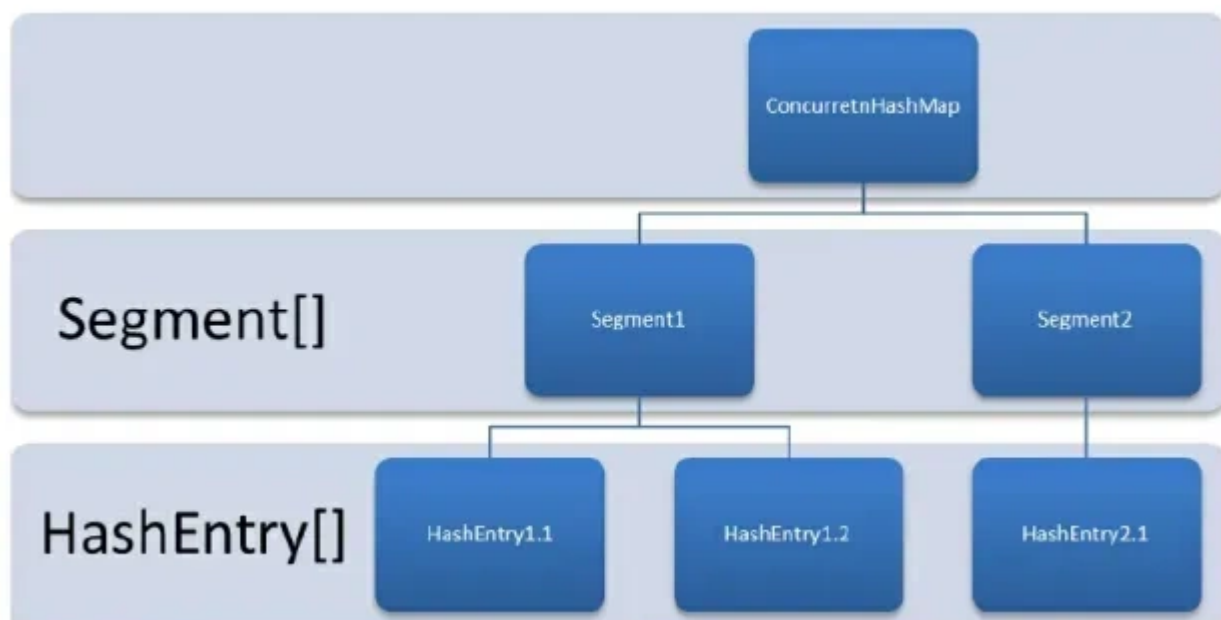
负载因子太低会导致大量的空桶浪费空间，负载因子太高会导致大量的碰撞，降低性能。0.75的负载因子在这两个因素之间取得了良好的平衡。

Hashmap和Hashtable有什么不一样的？Hashmap一般怎么用？

- **HashMap线程不安全**，效率高一点，可以存储null的key和value，null的key只能有一个，null的value可以有多个。默认初始容量为16，每次扩充变为原来2倍。创建时如果给定了初始容量，则扩充为2的幂次方大小。底层数据结构为数组+链表，插入元素后如果链表长度大于阈值（默认为8），先判断数组长度是否小于64，如果小于，则扩充数组，反之将链表转化为红黑树，以减少搜索时间。
- **HashTable线程安全**，效率低一点，其内部方法基本都经过synchronized修饰，不可以有null的key和value。默认初始容量为11，每次扩容变为原来的2n+1。创建时给定了初始容量，会直接用给定的大小。底层数据结构为数组+链表。它基本被淘汰了，要保证线程安全可以用ConcurrentHashMap。
- **怎么用：**HashMap主要用来存储键值对，可以调用put方法向其中加入元素，调用get方法获取某个键对应的值，也可以通过containsKey方法查看某个键是否存在等

ConcurrentHashMap怎么实现的？

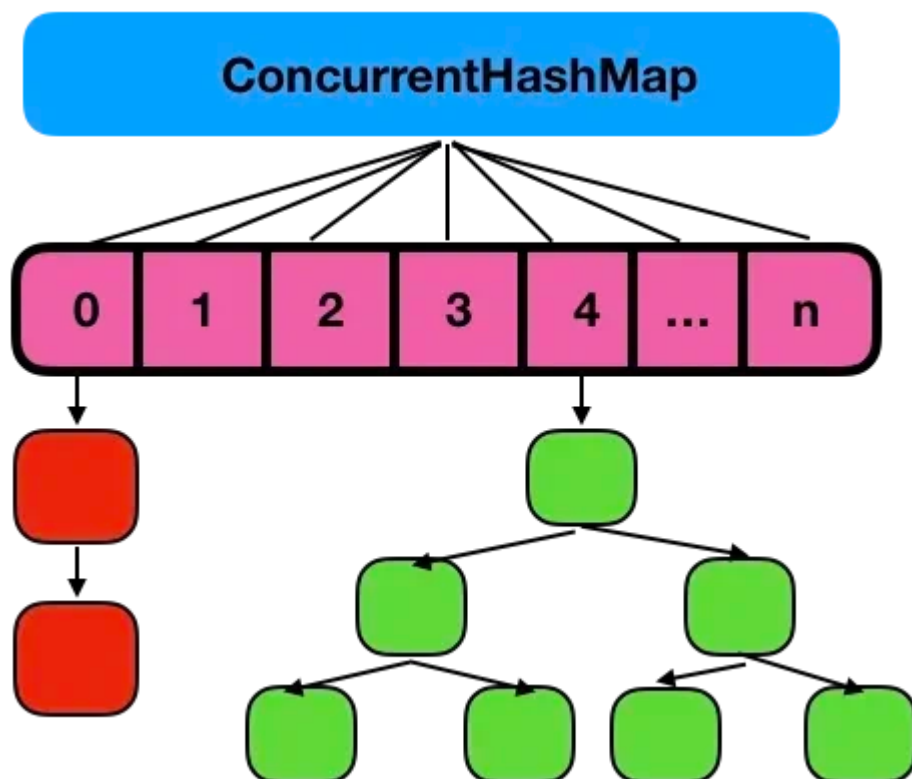
在 JDK 1.7 中它使用的是数组加链表的形式实现的，而数组又分为：大数组 Segment 和小数组 HashEntry。Segment 是一种可重入锁（ReentrantLock），在 ConcurrentHashMap 里扮演锁的角色；HashEntry 则用于存储键值对数据。一个 ConcurrentHashMap 里包含一个 Segment 数组，一个 Segment 里包含一个 HashEntry 数组，每个 HashEntry 是一个链表结构的元素。



JDK 1.7 ConcurrentHashMap 分段锁技术将数据分成一段一段的存储，然后给每一段数据配一把锁，当一个线程占用锁访问其中一个段数据的时候，其他段的数据也能被其他线程访问，能够实现真正的并发访问。

JDK 1.8 ConcurrentHashMap

在 JDK 1.7 中，ConcurrentHashMap 虽然是线程安全的，但因为它的底层实现是数组 + 链表的形式，所以在数据比较多的情况下访问是很慢的，因为要遍历整个链表，而 JDK 1.8 则使用了数组 + 链表/红黑树的方式优化了 ConcurrentHashMap 的实现，具体实现结构如下：



JDK 1.8 ConcurrentHashMap JDK 1.8 ConcurrentHashMap 主要通过 volatile + CAS 或者 synchronized 来实现的线程安全的。添加元素时首先会判断容器是否为空：

- 如果为空则使用 volatile 加 CAS 来初始化
- 如果容器不为空，则根据存储的元素计算该位置是否为空。
 - 如果根据存储的元素计算结果为空，则利用 CAS 设置该节点；
 - 如果根据存储的元素计算结果不为空，则使用 synchronized，然后，遍历桶中的数据，并替换或新增节点到桶中，最后再判断是否需要转为红黑树，这样就能保证并发访问时的线程安全了。

如果把上面的执行用一句话归纳的话，就相当于ConcurrentHashMap通过对头结点加锁来保证线程安全的，锁的粒度相比 Segment 来说更小了，发生冲突和加锁的频率降低了，并发操作的性能就提高了。

而且 JDK 1.8 使用的是红黑树优化了之前的固定链表，那么当数据量比较大的时候，查询性能也得到了很大的提升，从之前的 $O(n)$ 优化到了 $O(\log n)$ 的时间复杂度。

分段锁怎么加锁的？

在 ConcurrentHashMap 中，将整个数据结构分为多个 Segment，每个 Segment 都类似于一个小的 HashMap，每个 Segment 都有自己的锁，不同 Segment 之间的操作互不影响，从而提高并发性能。

在 ConcurrentHashMap 中，对于插入、更新、删除等操作，需要先定位到具体的 Segment，然后再在该 Segment 上加锁，而不是像传统的 HashMap 一样对整个数据结构加锁。这样可以使得不同 Segment 之间的操作并行进行，提高了并发性能。

分段锁是可重入的吗？

JDK 1.7 ConcurrentHashMap中的分段锁是用了 ReentrantLock，是一个可重入的锁。

已经用了synchronized，为什么还要用CAS呢？

ConcurrentHashMap使用这两种手段来保证线程安全主要是一种权衡的考虑，在某些操作中使用synchronized，还是使用CAS，主要是根据锁竞争程度来判断的。

比如：在putVal中，如果计算出来的hash槽没有存放元素，那么就可以直接使用CAS来进行设置值，这是因为在设置元素的时候，因为hash值经过了各种扰动后，造成hash碰撞的几率较低，那么我们可以预测使用较少的自旋来完成具体的hash落槽操作。

当发生了hash碰撞的时候说明容量不够用了或者已经有大量线程访问了，因此这时候使用synchronized来处理hash碰撞比CAS效率要高，因为发生了hash碰撞大概率来说是线程竞争比较强烈。

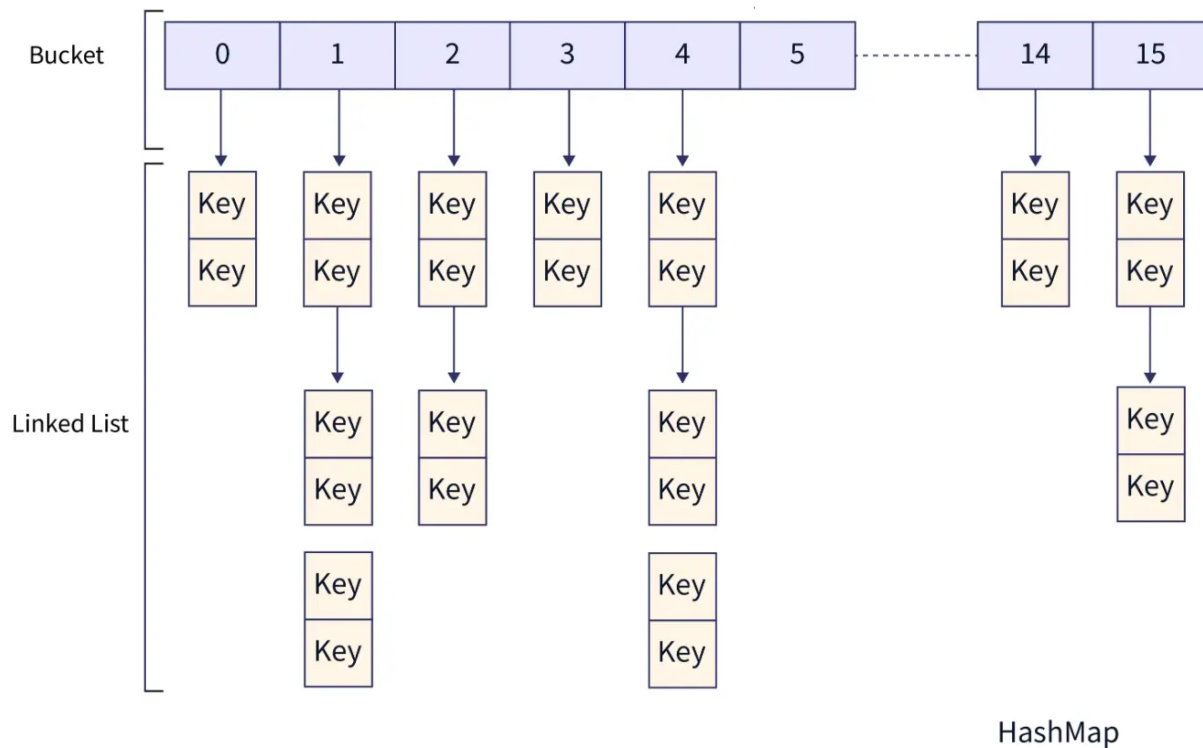
ConcurrentHashMap用了悲观锁还是乐观锁？

悲观锁和乐观锁都有用到。

添加元素时首先会判断容器是否为空：

- 如果为空则使用 volatile 加 **CAS（乐观锁）** 来初始化。
- 如果容器不为空，则根据存储的元素计算该位置是否为空。
- 如果根据存储的元素计算结果为空，则利用 **CAS（乐观锁）** 设置该节点；
- 如果根据存储的元素计算结果不为空，则使用 **synchronized（悲观锁）**，然后，遍历桶中的数据，并替换或新增节点到桶中，最后再判断是否需要转为红黑树，这样就能保证并发访问时的线程安全了。

HashTable 底层实现原理是什么？



- Hashtable的底层数据结构主要是**数组加上链表**，数组是主体，链表是解决hash冲突存在的。
- Hashtable是线程安全的，实现方式是**Hashtable的所有公共方法均采用synchronized关键字**，当一个线程访问同步方法，另一个线程也访问的时候，就会陷入阻塞或者轮询的状态。

Hashtable线程安全是怎么实现的？

因为它的put，get做成了同步方法，保证了Hashtable的线程安全性，每个操作数据的方法都进行同步控制之后，由此带来的问题任何一个时刻**只能有一个线程可以操纵Hashtable，所以其效率比较低。**

Hashtable 的 put(K key, V value) 和 get(Object key) 方法的源码：

```
public synchronized V put(K key, V value) {
    // Make sure the value is not null
    if (value == null) {
        throw new NullPointerException();
    }
    // Makes sure the key is not already in the hashtable.
    Entry,&gt; tab[] = table;
    int hash = key.hashCode();
    int index = (hash &amp; 0x7FFFFFFF) % tab.length;
    @SuppressWarnings("unchecked")
    Entry&lt;K,V&gt; entry = (Entry&lt;K,V&gt;)tab[index];
    for(; entry != null ; entry = entry.next) {
        if ((entry.hash == hash) &amp;&amp; entry.key.equals(key)) {
            V old = entry.value;
            entry.value = value;
            return old;
        }
    }
}</pre

```

```

    addEntry(hash, key, value, index);
    return null;
}

public synchronized V get(Object key) {
    Entry<?,?> tab[] = table;
    int hash = key.hashCode();
    int index = (hash & 0x7FFFFFFF) % tab.length;
    for (Entry<?,?> e = tab[index] ; e != null ; e = e.next) {
        if ((e.hash == hash) && e.key.equals(key)) {
            return (V)e.value;
        }
    }
    return null;
}

```

可以看到，**Hashtable**是通过使用了 **synchronized** 关键字来保证其线程安全。

在Java中，可以使用synchronized关键字来标记一个方法或者代码块，当某个线程调用该对象的synchronized方法或者访问synchronized代码块时，这个线程便获得了该对象的锁，其他线程暂时无法访问这个方法，只有等待这个方法执行完毕或者代码块执行完毕，这个线程才会释放该对象的锁，其他线程才能执行这个方法或者代码块。

hashtable 和concurrentHashMap有什么区别

底层数据结构：

- jdk7之前的ConcurrentHashMap底层采用的是**分段的数组+链表**实现，jdk8之后采用的是**数组+链表/红黑树**；
- Hashtable采用的是**数组+链表**，数组是主体，链表是解决hash冲突存在的。

实现线程安全的方式：

- jdk8以前，ConcurrentHashMap采用分段锁，对整个数组进行了分段分割，每一把锁只锁容器里的一部分数据，多线程访问不同数据段里的数据，就不会存在锁竞争，提高了并发访问；jdk8以后，直接采用数组+链表/红黑树，并发控制使用CAS和synchronized操作，更加提高了速度。
- Hashtable：所有的方法都加了锁来保证线程安全，但是效率非常的低下，当一个线程访问同步方法，另一个线程也访问的时候，就会陷入阻塞或者轮询的状态。

说一下HashMap和Hashtable、ConcurrentMap的区别

- HashMap线程不安全，效率高一点，可以存储null的key和value，null的key只能有一个，null的value可以有多个。默认初始容量为16，每次扩充变为原来2倍。创建时如果给定了初始容量，则扩充为2的幂次方大小。底层数据结构为数组+链表，插入元素后如果链表长度大于阈值（默认为8），先判断数组长度是否小于64，如果小于，则扩充数组，反之将链表转化为红黑树，以减少搜索时间。
- Hashtable线程安全，效率低一点，其内部方法基本都经过synchronized修饰，不可以有null的key和value。默认初始容量为11，每次扩容变为原来的2n+1。创建时给定了初始容量，会直接用给定的大小。底层数据结构为数组+链表。它基本被淘汰了，要保证线程安全可以用ConcurrentHashMap。
- ConcurrentHashMap是Java中的一个线程安全的哈希表实现，它可以在多线程环境下并发地进行读写操作，而不需要像传统的Hashtable那样在读写时加锁。ConcurrentHashMap的实现原理主要基于分段锁和CAS操作。它将整个哈希表分成了多Segment（段），每个Segment都类似于一个小的HashMap，它拥有自己的数组和一个独立的锁。在ConcurrentHashMap中，读操作不需要锁，可以直接对Segment进行读取，而写操作则只需要锁定对应的Segment，而不是整个哈希表，这样可以大大提高并发性能。

Set

Set集合有什么特点？如何实现key无重复的？

- **set集合特点**：Set集合中的元素是唯一的，不会出现重复的元素。
- **set实现原理**：Set集合通过内部的数据结构（如哈希表、红黑树等）来实现key的无重复。当向Set集合中插入元素时，会先根据元素的hashCode值来确定元素的存储位置，然后再通过equals方法来判断是否已经存在相同的元素，如果存在则不会再次插入，保证了元素的唯一性。

有序的Set是什么？记录插入顺序的集合是什么？

- **有序的 Set 是TreeSet和LinkedHashSet**。TreeSet是基于红黑树实现，保证元素的自然顺序。LinkedHashSet是基于双重链表和哈希表的结合来实现元素的有序存储，保证元素添加的自然顺序
- **记录插入顺序的集合通常指的是LinkedHashSet**，它不仅保证元素的唯一性，还可以保持元素的插入顺序。当需要在Set集合中记录元素的插入顺序时，可以选择使用LinkedHashSet来实现。

 面试鸭 mianshiya.com

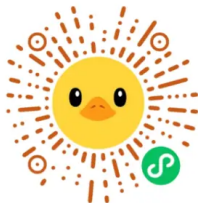
程序员面试 刷题神器

八股文一网打尽 面试从未如此轻松

- ✓ 9000+ 中大厂高频面试题
- ✓ 大厂面试官团队原创优质题解
- ✓ 网页版+小程序+IDE、VSCode插件
- ✓ 真题命中率高，持续更新

微信扫一扫 >>

刷高频题拿高薪offer



最新的图解文章都在公众号首发，别忘记关注哦！！如果你想加入百人技术交流群，扫码下方二维码回复「加群」。



扫一扫，关注「小林coding」公众号

图解计算机基础
认准**小林coding**

每一张图都包含小林的认真
只为帮助大家能更好的理解

- ① 关注公众号回复「**图解**」
获取图解系列 PDF
- ② 关注公众号回复「**加群**」
拉你进百人技术交流群